ENTREGA 1: PROPOSTA DO PROJETO IoT

Sensor de Alarme com Detector de Movimento (PIR)

Eduardo Octavio de Paula Souza

1. PROPOSTA DO PROJETO

2.1 Problema Identificado

Muitos ambientes (casas, escritórios, laboratórios) precisam de um sistema de segurança simples que detecta movimentos e dispara um alarme apenas em horários específicos. O problema é que alarmes tradicionais funcionam o tempo inteiro ou precisam ser ativados manualmente, o que é inconveniente. Uma solução IoT permite monitorar em tempo real e controlar o alarme remotamente via um painel web, definindo horários de operação automaticamente.

2.2 Objetivo

Desenvolver um sistema de alarme inteligente com detector de movimento que permite: detectar movimento através de um sensor PIR; disparar um buzzer quando movimento é detectado fora do horário permitido; acender um LED indicador quando movimento é detectado; controlar o alarme remotamente via painel web (ligar/desligar, ajustar horários); manter histórico de todas as detecções com timestamp.

2.3 Sensor Utilizado

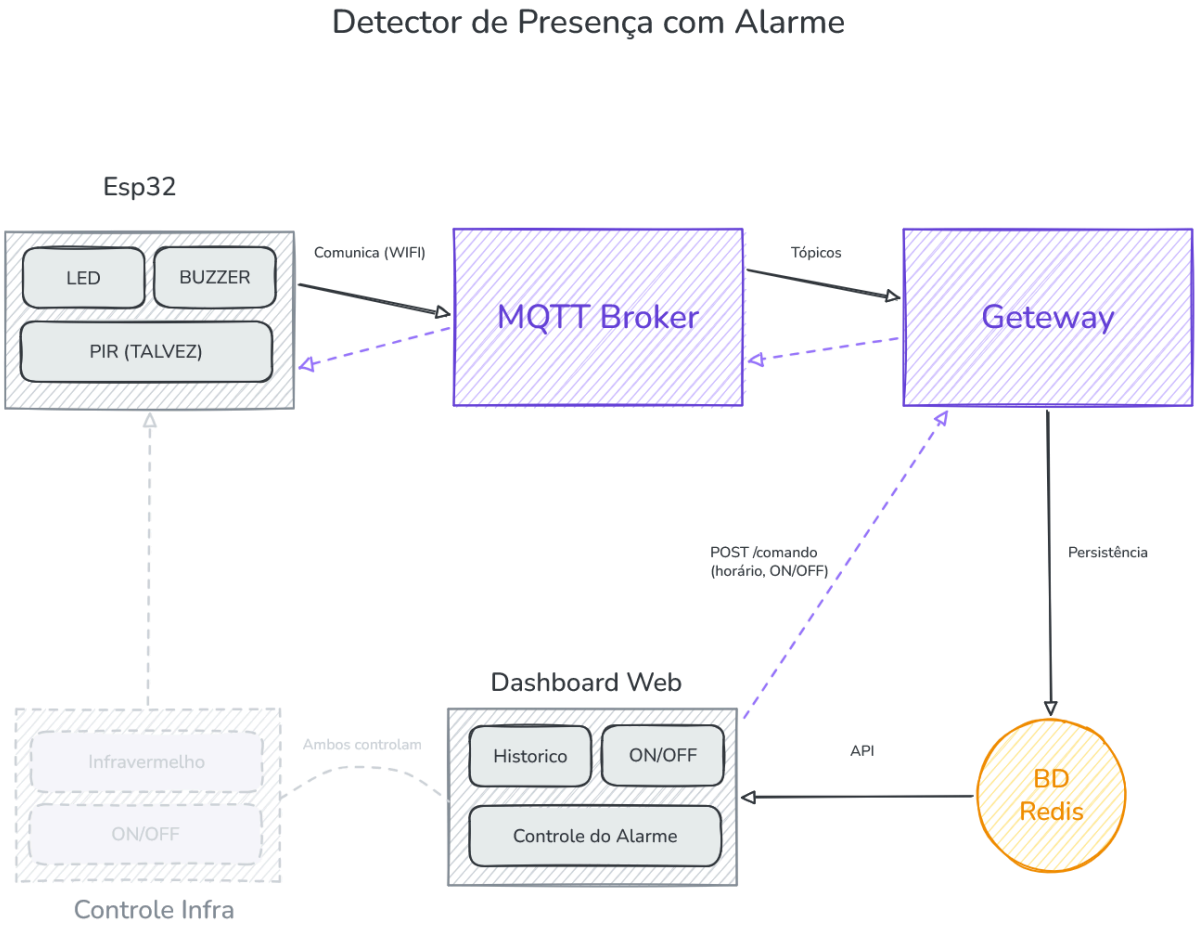
Sensor PIR HC-SR501: este sensor detecta radiação infravermelha emitida por corpos em movimento. Quando alguém passa perto dele, detecta a mudança de temperatura e gera um sinal digital (HIGH ou LOW). É perfeito para aplicações de alarme e segurança.

2.4 Atuadores Utilizados

Buzzer Ativo 5V: um pequeno alto-falante que emite um som de alarme quando recebe energia. Será controlado pelo ESP32 via GPIO.

LED: um indicador visual que acende quando movimento é detectado, fornecendo feedback imediato ao usuário.

3. DIAGRAMA DA ARQUITETURA



O diagrama acima mostra a integração entre os componentes do sistema: ESP32 recebe dados do sensor PIR via GPIO, conecta à rede Wi-Fi, publica mensagens MQTT no broker, o gateway Node.js processa estas mensagens e persiste em Redis, e o painel web exibe os dados e envia comandos de controle de volta ao sistema.

4. FLUXO INICIAL DE DADOS E MENSAGENS

4.1 Fluxo

1. O sensor PIR detecta movimento.
2. ESP32 lê o sinal e cria uma mensagem JSON.
3. ESP32 publica a mensagem no tópico MQTT.
4. Gateway Node.js assina o tópico e recebe a mensagem.
5. Gateway valida a mensagem e é salva em Redis.
6. O Painel web faz requisição GET /api/status e recebe os dados.

4.2 Fluxo de Controle

1. Usuário move o slider de horário no painel web
2. Painel web envia POST /api/comando
3. Gateway recebe o comando e salva a configuração em Redis.

4. Gateway pública no tópico MQTT

5. ESP32 assina tópico alarme/config, recebe mensagem e atualiza sua configuração local.

6. A partir disso, quando o PIR detecta movimento fora do horário, buzzer e LED são ativados automaticamente.

5. CONSIDERAÇÕES FINAIS

Acredito que este projeto é viável, relevante e alinha-se bem com os objetivos da disciplina. Integra componentes reais, envolve conectividade, processamento em Gateway e interface web. Espero não apenas construir um sistema funcional, mas compreender profundamente cada camada desta arquitetura e documentar todas as decisões técnicas tomadas.